CSA1220 Computer Architecture

1. Debugging Signed Binary Subtraction Using 2's Complement

Problem

A computer system performing signed binary subtraction using the 2's complement method produces incorrect results for certain combinations of positive and negative operands. The errors may occur due to incorrect complement generation, sign extension, or failure to detect overflow.

Debugging Process

1. Verify that both operands are represented in the same number of bits.
2. Convert the subtrahend into its **2's complement** by taking the 1's complement and adding 1.
3. Check whether sign extension is correctly applied before performing subtraction.
4. Perform binary addition between the minuend and the 2's complement of the subtrahend.
5. Ignore the final carry-out for signed arithmetic operations.
6. Detect overflow by comparing the carry into and carry out of the most significant bit (MSB). If they differ, overflow has occurred.
7. Test the circuit using positive-positive, positive-negative, negative-positive, and negative-negative operand combinations.

Suggested Corrections

- Implement correct 2's complement generation hardware.
- Ensure proper sign extension for operands with different bit lengths.
- Add overflow detection circuitry using XOR of the carry into and carry out of the MSB.
- Validate the subtraction process using simulation and corner-case testing.

Result

The corrected system performs accurate signed subtraction for all operand combinations while correctly identifying arithmetic overflow conditions.

2. Debugging a Carry Look-Ahead Adder (CLA)

Problem

A Carry Look-Ahead Adder fails to generate the correct sum for certain input combinations because of errors in the generate and propagate logic or carry computation network.

Debugging Process

1. Verify the Generate signal:
   [
   G_i = A_i \cdot B_i
   ]
2. Verify the Propagate signal:
   [
   P_i = A_i \oplus B_i
   ]
3. Check each carry equation:
   [
   C_{i+1}=G_i+P_iC_i
   ]
4. Examine all logic gate connections for incorrect wiring or missing carry paths.
5. Compare the CLA output with a Ripple Carry Adder using the same test inputs.
6. Test all possible carry propagation conditions.

Suggested Modifications

- Correct any faulty generate or propagate logic.
- Optimize carry-generation equations to minimize gate delay.
- Reduce fan-out using buffering if necessary.
- Verify timing constraints using hardware simulation.
- Perform exhaustive testing for all possible input combinations.

Result

The corrected CLA restores high-speed addition with accurate carry generation while maintaining the low propagation delay that makes CLA faster than ripple carry adders.

3. Debugging Booth's Multiplication Algorithm

Problem

A hardware multiplier implementing Booth's algorithm consumes more clock cycles than expected during signed multiplication, reducing overall processor performance.

Debugging Process

1. Verify initialization of the accumulator (A), multiplier (Q), multiplicand (M), and extra bit (Q-1).
2. Check the Booth decision rules:
    - 00 → No operation
    - 11 → No operation
    - 01 → Add multiplicand
    - 10 → Subtract multiplicand
3. Verify that arithmetic right shifts preserve the sign bit.
4. Ensure register updates occur after every iteration.
5. Confirm that the iteration count equals the number of bits in the multiplier.
6. Compare execution with manual multiplication results.

Optimization Techniques

- Use Radix-4 Booth encoding to reduce the number of iterations by half.
- Skip unnecessary addition and subtraction operations.
- Optimize control logic for faster state transitions.
- Use pipelining to overlap multiplication stages.
- Employ parallel hardware where feasible.

Result

These improvements significantly reduce execution time while preserving the accuracy of signed multiplication operations.

4. Debugging the Non-Restoring Division Algorithm

Problem

A processor implementing the non-restoring division algorithm produces incorrect quotient values because of errors in partial remainder updates, shift operations, or quotient-bit generation.

Debugging Process

1. Verify initialization of dividend, divisor, accumulator, and quotient registers.
2. Check left-shift operations after each iteration.
3. Ensure subtraction is performed when the partial remainder is positive and addition when it is negative.
4. Verify the generation of quotient bits after each arithmetic operation.
5. Monitor the partial remainder after every iteration.
6. Perform final correction if the remainder is negative.
7. Validate the algorithm with multiple dividend-divisor combinations.

Suggested Corrections

- Correct the arithmetic control logic.
- Ensure proper synchronization between shifting and arithmetic operations.
- Implement accurate quotient-bit assignment.
- Add hardware to restore the final remainder when necessary.
- Verify all intermediate calculations using simulation tools.

Result

The corrected non-restoring divider produces accurate quotient and remainder values while maintaining efficient hardware utilization.

5. Debugging IEEE 754 Double-Precision Floating-Point Computations

Problem

A graphics processing application experiences slow execution because it performs a large number of floating-point calculations using IEEE 754 double-precision representation, even when high precision is unnecessary.

Debugging Process

1. Profile the application to identify floating-point intensive sections.
2. Check whether double precision is required for every computation.
3. Detect unnecessary conversions between integer, single precision, and double precision.
4. Identify repeated calculations inside loops.
5. Measure execution time and memory usage of floating-point operations.
6. Analyze compiler optimization reports for inefficient arithmetic instructions.

Optimization Techniques

- Replace double precision with single precision wherever acceptable.
- Eliminate redundant floating-point calculations through common sub-expression elimination.
- Use SIMD/vector instructions for parallel floating-point operations.
- Enable compiler optimization flags for floating-point arithmetic.
- Utilize hardware Floating Point Units (FPUs) and GPU acceleration.
- Cache frequently used values to reduce repeated computations.

Result

The optimized implementation reduces execution time, lowers memory bandwidth usage, and improves graphics performance while maintaining the required numerical accuracy and IEEE 754 compliance.